



Trabajo de Fin de Grado

Grado Superior Doble en Desarrollo de Aplicaciones Web y Multiplataforma

JHTTP

Desarrollo de un servidor HTTP/HTTPS en Java.

Autor: *Sergio de Santiago Redondo*

Tutor: *Daniel Sierra Solis*

Curso: *2025/2026*

Í N D I C E

INTRODUCCIÓN

1.1. Descripción.....	2
-----------------------	---

ESTRUCTURA DEL PROYECTO

2.1. Descripción.....	3
-----------------------	---

ESTADO ACTUAL DEL PROYECTO

3.1. Arranque del programa.....	4
3.2. Flujo de la conexión.....	5
3.3. Gestión de la respuesta saliente.....	7

CAPÍTULO 1

INTRODUCCIÓN

1.1. Descripción

El presente documento tiene como objetivo mostrar el progreso en el desarrollo del Trabajo de Fin de Grado JHTTP, siendo este el segundo documento presentado con este fin.

CAPÍTULO 2

ESTRUCTURA DEL PROYECTO

2.1. Descripción

La aplicación está dividida, por el momento, en 7 paquetes diferenciados, cada uno con su respectiva función, frente a los 6 paquetes presentes en la anterior versión.

- **core:** Alberga el núcleo de la aplicación. En él, residen las clases **Acceptor**, la cual gestiona las conexiones entrantes; y **Executor**, que gestiona independientemente el contexto de cada conexión.
- **http:** Gestiona el procesamiento de los mensajes HTTP. Contiene las clases **HttpRequest** y **HttpResponse** actuando como entidades principales, y **HttpStatus**, **HttpVersion**, **HttpMethod** y **HttpHeaders** siendo utilizadas dentro de estas. La creación de ambas entidades principales es gestionada a través de su correspondiente clase *builder*.

Los paquetes no mencionados no han sufrido cambios con respecto a la versión anterior.

CAPÍTULO 3

ESTADO ACTUAL DEL PROYECTO

3.1. Arranque del programa

Se gestiona el uso del puerto indicado por el usuario, en el ejemplo el 8080, se configura la gestión por parte del socket del servidor y se arrancan los componentes principales, **Acceptor** y **Executor**. Hecho esto, se da inicio al funcionamiento del servidor.

```
public void start() throws Exception { 1 usage  👤 Sergio *
    printBanner();

    serverSocket = new ServerSocket();

    final var address = new InetSocketAddress(config.getPort());
    serverSocket.bind(address);
    log.info(message: "Server socket bound to port '{}'", serverSocket.getLocalPort());

    final var executor = new Executor(createPipeline());

    final var acceptor = new Acceptor(serverSocket, executor);
    Thread.ofPlatform()
        .name("acceptor")
        .daemon(false)
        .start(acceptor);
}
```

Cabe destacar, con respecto a la anterior versión, el uso de **SocketServer** con respecto a **SocketServerChannel**. Esto se debe al uso, en esta nueva versión, de **VirtualThreads**, introducidos en Java 21; frente a los **PlatformThreads** tradicionales. Este cambio permite desechar la clase **Worker** para crear un hilo virtual por cada conexión, simplificando en gran medida la gestión de cada conexión.

```
public Executor(final @NonNull Pipeline pipeline) { 1 usage
    this.pipeline = pipeline;

    final var virtualThreadFactory = Thread.ofVirtual()
        .name("executor")
        .factory();
    this.executorService = Executors.newThreadPerTaskExecutor(virtualThreadFactory);
}
```

Sirvan los logs como muestra del que el proceso se lleva a cabo con éxito.

```

  / / / / / _ / / / \
 / / / / / / / / / /
/ / / / / _ / / / /
\ / / / / / / / / /
2026-05-22 23:38:02.962+02:00 INFO --- [main] sdevsantiago.jhttp.config.ServerBootstrap : Server socket bound to port '8080'
2026-05-22 23:38:02.972+02:00 INFO --- [acceptor] sdevsantiago.jhttp.core.Acceptor : Server running
```

3.2. Flujo de la conexión

Una vez aceptada la petición por el **Acceptor**, se crea un hilo en el **Executor** y se manda la tarea a este.

```
@Override Sergio *
public void run() {
    log.info("Server running");

    while (!Thread.currentThread().isInterrupted()) {
        try {
            final var socket = serverSocket.accept();

            final var address = socket.getInetAddress();
            log.info(message: "Accepted connection from {}", address);

            executor.getExecutorService().submit(() → {
                executor.handleConnection(socket);
            });
        } catch (final IOException e) {
            log.error(message: "Error handling connection", e);
        }
    }
}
```

```
public void handleConnection(final @NonNull Socket socket) { 1 usage
    try {
        pipeline.execute(new Context(socket));
    } catch (final Exception e) {
        log.fatal(message: "Unhandled exception", e);

        try {
            HttpResponseEncoder.sendResponse(
                socket.getOutputStream(),
                HttpResponse.internalServerError().build()
            );
        } catch (final IOException ioException) {
            log.debug(message: "Failed to send error response", ioException);
        }
    } finally {
        closeSocket(socket);
    }
}
```

La función que cumplía el **Worker** en este estado contenía errores a la hora de cerrar el socket, pues esto no se realizaba correctamente y el servidor llegaba a saturarse con frecuencia. Esta nueva versión es capaz de una mayor cantidad de peticiones sin problemas, llegando a registrarse más de 500.000 peticiones exitosas en menos de 5 minutos.

```
private Pipeline createPipeline() { 1 usage new *
    return new Pipeline()
        .addHandler(new HttpRequestDecoder())
        .addHandler(new Router())
        .addHandler(new HttpResponseEncoder());
}
```

```
public void execute(final Context context) { 1 usage
    for (final var inboundHandler : handlers) {
        inboundHandler.inbound(context);

        if (contextIsTerminated(context)) {
            return;
        }
    }

    for (final var outboundHandler : handlers.reversed()) {
        outboundHandler.outbound(context);

        if (contextIsTerminated(context)) {
            return;
        }
    }
}
```

El **ServerBootstrap** es, por el momento, quien define el flujo de etapas de la conexión, siendo este sencillo por el momento. Esta nueva versión permite añadir y eliminar etapas sin que esto afecte al código.

3.3. Gestión de la respuesta saliente

Cuando el tick está en modo lectura, se comienza a escribir en el socket del cliente. Una vez terminado, se elimina dicha clave y se da por cerrada finalizado el intercambio de mensajes. Entre las posibles cabeceras, o headers, que se pueden recibir en el mensaje entrante, existe la cabecera *Connection*, con valor posible *keep-alive*. Esto indica si la petición se ha llegado a completar por parte del cliente. Este comportamiento no está aún implementado.

```
@Override 1 usage
public void outbound(final @NonNull Context context) {
    try {
        sendResponse(
            context.getSocket().getOutputStream(),
            context.getResponse()
        );
    } catch (final IOException ioException) {
        log.debug( message: "Response error: ", ioException);
    }
}
```

```
public static void sendResponse(final @NonNull OutputStream outputStream, 2 usages
                                final @NonNull HttpResponse response) throws IOException {
    outputStream.write(response.statusLine().getBytes(StandardCharsets.UTF_8));
    outputStream.write(LINE_SEPARATOR);

    for (final var header : response.headers().entrySet()) {
        final var headerLine = StringUtils.concatWith( delimiter: " ", header.getKey(), header.getValue().toString());
        outputStream.write(headerLine.getBytes(StandardCharsets.UTF_8));

        outputStream.write(LINE_SEPARATOR);
    }

    outputStream.write(LINE_SEPARATOR);

    outputStream.write(response.body());

    log.debug( message: "Response sent: {}", response.status().format());
}
```

Estado	Método	Domino	Archivo	Iniciador	Tipo	Transferido	Tamaño	ms
200	GET	localhost:8080	/	document	plain		19 B	0 B 4 ms
404	GET	localhost:8080	/favicon.ico	img	plain		26 B	0 B 7 ms